

NAME: K. Lahari

REG NO: 192325118

COURSE : ITA0515 - COMPUTER VISION

SLOT : B

EXPERIMENT-5

Aim: To read an image in Python and perform erosion using the OpenCV `erode()` function.

Algorithm

1. Import OpenCV and NumPy libraries.
2. Read the input image.
3. Create a kernel matrix.
4. Apply `cv2.erode()` to the image.
5. Display the original and eroded images.

CODE:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
img = cv2.imread(r"/content/images.png")
kernel=np.ones((5,5),np.uint8)
erode=cv2.erode(img,kernel,iterations=1)
cv2_imshow(img)
cv2_imshow(erode)
```



Result

The image was successfully read and erosion was performed. Small white regions were removed, and the object boundaries became thinner.

EXPERIMENT-6

Aim

To read a captured video and display it in slow motion and fast motion using OpenCV.

Algorithm

1. Import OpenCV.
2. Open the video using VideoCapture().
3. Read each frame.
4. Display the frames with different delay times using waitKey().
5. Close the video after completion.

CODE:

```
import cv2

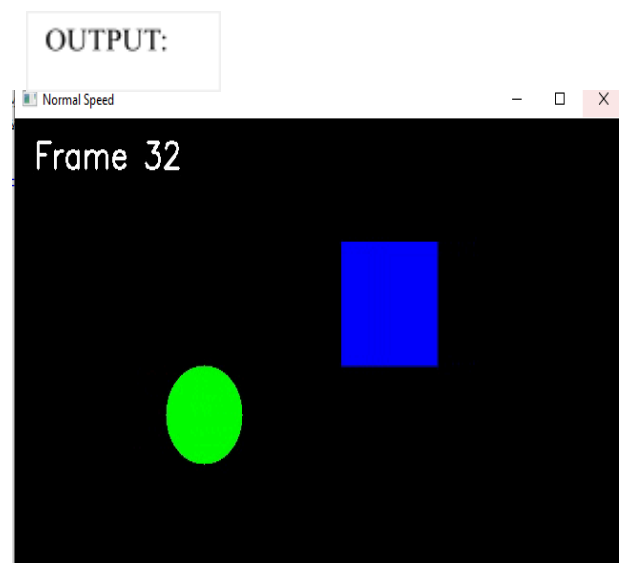
from google.colab.patches import cv2_imshow

path = r"/content/istockphoto-2155009603-640_adpp_is.mp4"

def play_video(delay, title):
    cap = cv2.VideoCapture(path)
    print(title + " - Press q to continue")
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        cv2_imshow(frame)
        if cv2.waitKey(delay) & 0xFF == ord('q'):
            break
    cap.release()

# Call function for different speeds
play_video(30, "Normal Speed")
play_video(100, "Slow Motion")
play_video(10, "Fast Motion")
```

RESULT- The program is executed successfully.



EXPERIMENT-7

Aim

To capture video from a webcam and display it in slow motion and fast motion.

Algorithm

1. Import OpenCV.
2. Access the webcam using VideoCapture(0).
3. Capture video frames continuously.
4. Display frames using different delay values.
5. Exit when a key is pressed.

CODE:

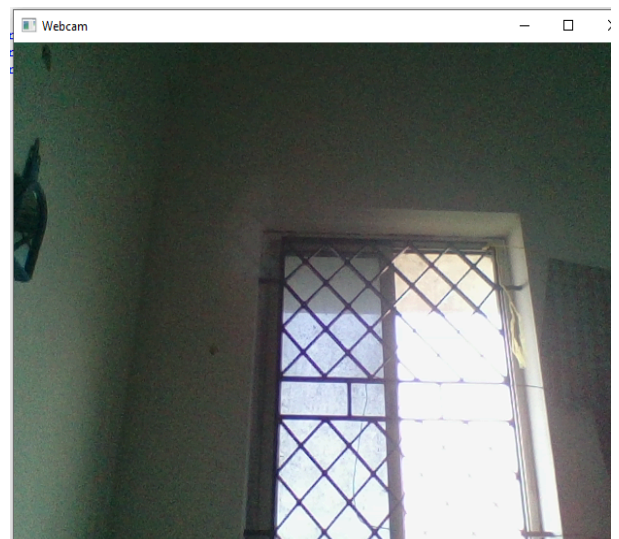
```
import cv2

from google.colab.patches import cv2_imshow

path = r"/content/istockphoto-2155009603-640_adpp_is.mp4"

def play_video(delay, title):
    cap = cv2.VideoCapture(path)
    print(title + " - Press q to continue")
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        cv2_imshow(frame)
        if cv2.waitKey(delay) & 0xFF == ord('q'):
            break
    play_video(30, "Normal Speed")
    play_video(100, "Slow Motion")
    play_video(10, "Fast Motion")
```

OUTPUT:



RESULT- The program is executed successfully.

EXPERIMENT-8

Aim

To scale an image into bigger and smaller sizes using OpenCV.

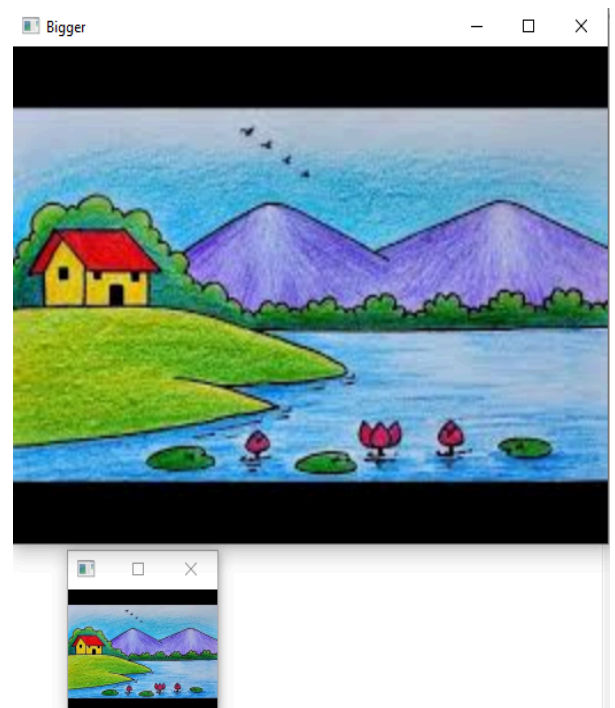
Algorithm

1. Read the input image.
2. Use cv2.resize() with scale factors.
3. Create enlarged and reduced images.
4. Display all images.

CODE:

```
import cv2
from google.colab.patches import cv2_imshow
Upload the image to Colab first
image = cv2.imread("/content/images.png")
if image is None:
    print("Image not found!")
else:
    bigger = cv2.resize(image, None, fx=2, fy=2)
    smaller = cv2.resize(image, None, fx=0.5, fy=0.5)
    print("Original Image")
    cv2_imshow(image)
    print("Bigger Image")
    cv2_imshow(bigger)
    print("Smaller Image")
    cv2_imshow(smaller)
```

OUTPUT:



RESULT- The program is executed successfully.

EXPERIMENT-9

Aim

To rotate an image in clockwise and counter-clockwise directions.

Algorithm

1. Read the image.
2. Use cv2.rotate() for clockwise rotation.
3. Use cv2.rotate() for counter-clockwise rotation.
4. Display the rotated images.

CODE:

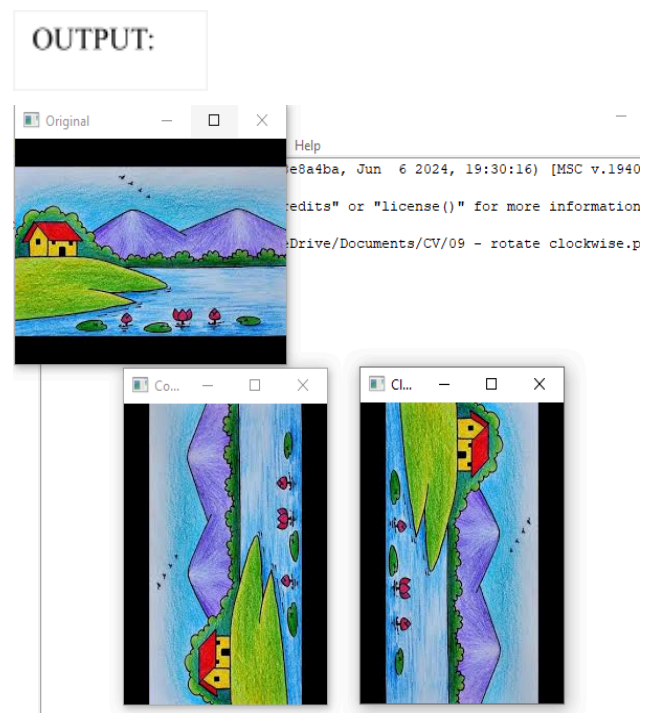
```
import cv2

from google.colab.patches import cv2_imshow

image = cv2.imread("/content/images.png")

if image is None:
    print("Image not found!")
else:
    clockwise = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)
    counter_clockwise = cv2.rotate(image, cv2.ROTATE_90_COUNTERCLOCKWISE)
    print("Original Image")
    cv2_imshow(image)
    print("Clockwise Rotation")
    cv2_imshow(clockwise)
    print("Counter-Clockwise Rotation")
    cv2_imshow(counter_clockwise)
```

RESULT- The program is executed successfully.



EXPERIMENT-10

Aim

To move an image from one position to another using translation.

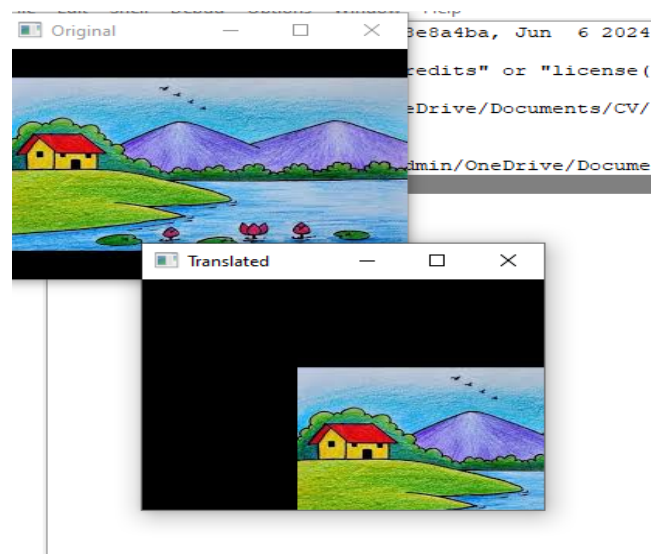
Algorithm

1. Read the image.
2. Create a translation matrix.
3. Apply `cv2.warpAffine()`.
4. Display the translated image.

CODE:

```
import cv2
import numpy as np
from google.colab import files
from google.colab.patches import cv2_imshow
uploaded = files.upload()
image = cv2.imread("sample.jpg")
rows, cols = image.shape[:2]
M = np.float32([[1,0,100],
                [0,1,50]])
move = cv2.warpAffine(image, M, (cols, rows))
print("Original")
cv2_imshow(image)
print("Moved")
cv2_imshow(move)
```

OUTPUT:



RESULT- The program is executed successfully.

EXPERIMENT-11

Aim

To perform affine transformation on an image using OpenCV.

Algorithm

1. Read the image.
2. Select three source points.
3. Select three destination points.
4. Compute the affine transformation matrix.
5. Apply `cv2.warpAffine()`.
6. Display the transformed image.

CODE:

```
import cv2
import numpy as np
from google.colab import files
from google.colab.patches import cv2_imshow
uploaded = files.upload()
image = cv2.imread("sample.jpg")
rows, cols = image.shape[:2]
pts1 = np.float32([[50,50],[200,50],[50,200]])
pts2 = np.float32([[10,100],[200,50],[100,250]])
M = cv2.getAffineTransform(pts1, pts2)
affine = cv2.warpAffine(image, M, (cols, rows))
print("Original")
cv2_imshow(image)
print("Affine")
cv2_imshow(affine)
```

RESULT- The program is executed successfully.



EXPERIMENT-12

Aim

To perform perspective transformation on an image using OpenCV.

Algorithm

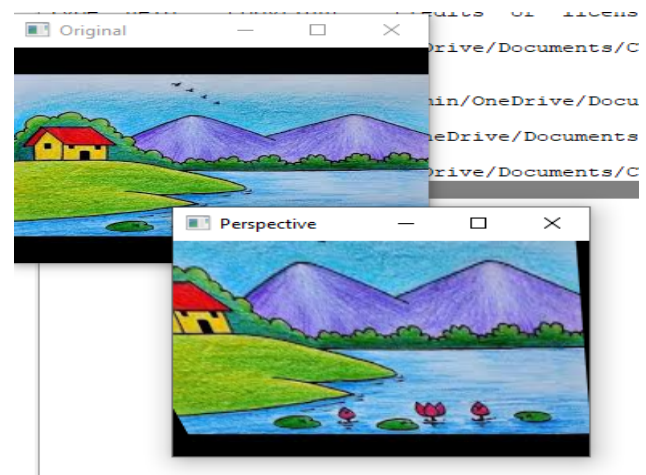
1. Read the input image.
2. Select four source points.
3. Select four destination points.
4. Compute the perspective transformation matrix.
5. Apply `cv2.warpPerspective()`.
6. Display the transformed image.

CODE:

```
import cv2
import numpy as np
from google.colab import files
from google.colab.patches import cv2_imshow
uploaded = files.upload()
image = cv2.imread("sample.jpg")
rows, cols = image.shape[:2]
pts1 = np.float32([[50,50],[300,50],[50,300],[300,300]])
pts2 = np.float32([[10,100],[300,50],[100,250],[280,300]])
M = cv2.getPerspectiveTransform(pts1, pts2)
perspective = cv2.warpPerspective(image, M, (cols,
rows))
print("Original")
cv2_imshow(image)
print("Perspective")
cv2_imshow(perspective)
```

RESULT- The program is executed successfully.

OUTPUT:



EXPERIMENT-13

Aim

To perform perspective transformation on a video using OpenCV.

Algorithm

1. Read the video.
2. Capture each video frame.
3. Define source and destination points.
4. Compute the perspective transformation matrix.
5. Apply `cv2.warpPerspective()` to each frame.
6. Display the transformed video.

CODE:

```
import cv2
import numpy as np
cap = cv2.VideoCapture(r"C:\Users\admin\Downloads\sample_video.mp4")
if not cap.isOpened():
    print("Error: Cannot open video file.")
    exit()
while True:
    ret, frame = cap.read()
    if not ret:
        print("Video Finished")
        break
    h, w = frame.shape[:2]
    pts1 = np.float32([
        [50, 50],
        [w - 50, 50],
        [50, h - 50],
        [w - 50, h - 50]
    ])
    pts2 = np.float32([
        [0, 0],
```

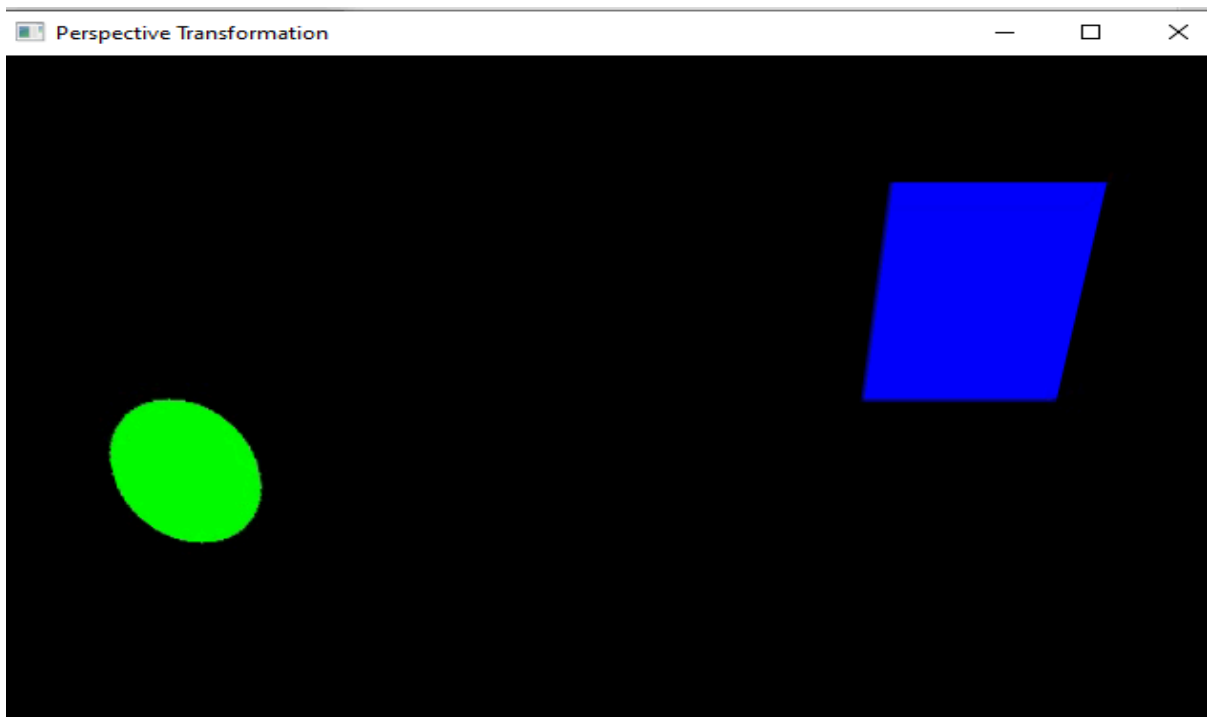
```

[w, 0],
[100, h],
[w - 100, h]
])
M = cv2.getPerspectiveTransform(pts1, pts2)
output = cv2.warpPerspective(frame, M, (w, h))
cv2.imshow("Original Video", frame)
cv2.imshow("Perspective Transformation", output)
if cv2.waitKey(30) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

OUTPUT:



RESULT- The program is executed successfully.

EXPERIMENT - 14

Aim

To perform image transformation using a Homography matrix in OpenCV.

Algorithm

1. Read the input image.
2. Select source and destination points.
3. Compute the Homography matrix.
4. Apply the transformation using `warpPerspective()`.
5. Display the transformed image.

CODE:

```
import cv2

import numpy as np

img = cv2.imread(r"C:\Users\admin\Downloads\image.jfif")

rows, cols = img.shape[:2]

src = np.float32([[50,50], [300,50], [50,300], [300,300]])

dst = np.float32([[20,80], [280,40], [80,320], [320,280]])

H, status = cv2.findHomography(src, dst)

result = cv2.warpPerspective(img, H, (cols, rows))

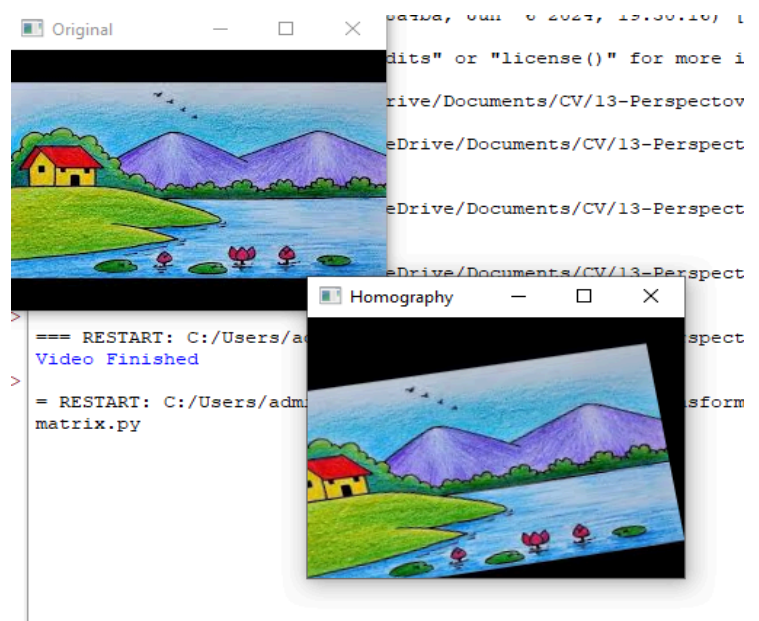
cv2.imshow("Original", img)

cv2.imshow("Homography", result)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT:



RESULT- The program is executed successfully.

EXPERIMENT - 15

Aim

To perform image transformation using the Direct Linear Transformation (DLT) method in OpenCV.

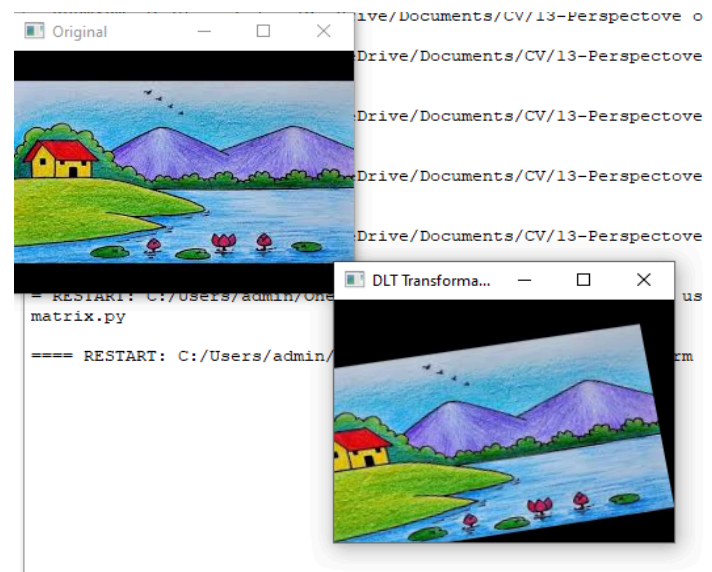
Algorithm

1. Read the input image.
2. Define the source and destination points.
3. Compute the transformation matrix using the DLT method.
4. Apply the perspective transformation.
5. Display the transformed image.

CODE:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\admin\Downloads\image.jfif")
rows, cols = img.shape[:2]
src = np.float32([[50,50], [300,50], [50,300], [300,300]])
dst = np.float32([[20,80], [280,40], [80,320], [320,280]])
# DLT is internally used by OpenCV to compute the homography
H = cv2.getPerspectiveTransform(src, dst)
result = cv2.warpPerspective(img, H, (cols, rows))
cv2.imshow("Original", img)
cv2.imshow("DLT Transformation", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



RESULT- The program is executed successfully